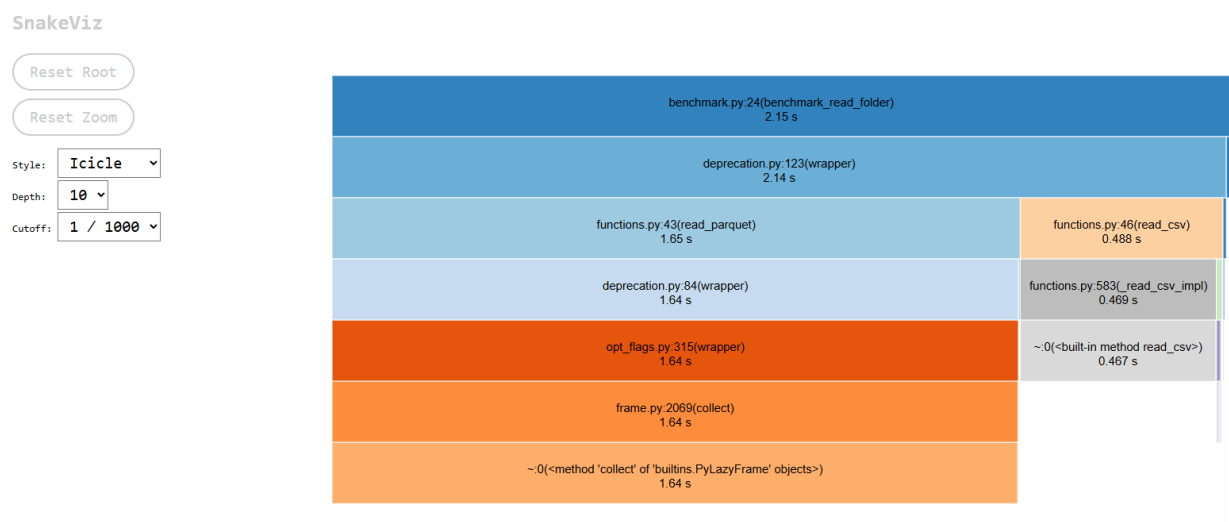


Challenging the Parquet Dogma

A Benchmark Report on CSV vs Parquet
Performance in Real-world Analytics



*A Real-World Performance Report Comparing CSV and Parquet in Local
Analytics Workflows*

Executive Summary	3
Background & Motivation	4
Benchmark Setup	5
Benchmark Results	6
Profiling Analysis	8
Insights & Discussion	10
Recommendation & Best Practices	12
Appendix A. Hardware & Environment Specs	14
Appendix B. Benchmark Script Snippet	15
Appendix C. On-Disk File Size Comparison Between CSV and Parquet Formats	16

Executive Summary

In data engineering and analytics, it is commonly assumed that Parquet file formats outperform CSV due to their columnar storage and compression capabilities. However, this report challenges that assumption by presenting a benchmarking study conducted in a local analytics environment using 50 identically structured files across both formats.

Contrary to expectations, CSV files demonstrated faster load times and significantly lower memory usage than Parquet files, especially in small-file, warm-cache scenarios. Detailed profiling revealed that Polars' lazy evaluation model for Parquet incurs overhead through `.collect()` operations, which does not occur when reading CSV eagerly.

This finding highlights that while Parquet remains superior for large-scale, columnar, or distributed processing, CSV may offer better performance in localized, interactive analysis tasks—particularly when working with multiple small files.

Based on these results, we recommend re-evaluating file format choices not based on general assumptions but on context-specific performance profiling.

Background & Motivation

File format decisions in data analytics are often made based on general assumptions or community norms—most notably, that Parquet is universally faster and more efficient than CSV. While this belief holds true in many large-scale, distributed processing contexts, it may not be valid for all use cases.

In localized environments, especially when working with many small files or conducting quick exploratory analysis, the characteristics that make Parquet efficient (e.g., columnar compression, metadata overhead, lazy evaluation) may introduce unintended latency and memory overhead.

This project began with a simple question: *Is Parquet always the better choice for speed and efficiency?* As a developer regularly navigating real-world analytics pipelines, I wanted to understand whether this assumption holds when measured under realistic, local processing conditions.

Benchmark Setup

To evaluate the performance of CSV and Parquet file formats in a local analytics environment, this study employed a controlled benchmarking script built with Python and the Polars library. The goal was to measure differences in file loading time and memory consumption across both formats under reproducible conditions.

The dataset consisted of 50 identically structured files in each format, each containing the same schema, row count, and content. The files were stored locally and loaded using the same Polars function (``read_csv()`` or ``read_parquet()``) within a looped benchmarking routine.

Performance was measured under two cache conditions:

- **Cold Cache: System rebooted before loading, to eliminate file system caching effects.**
- **Warm Cache: Files loaded repeatedly without reboot, to simulate real-world repeated access.**

The benchmarking script recorded both **execution time** and **virtual memory usage (VMS)** for each run. Additionally, Python's ``cProfile`` module and ``SnakeViz`` were used to visualize function-level runtime behavior. System specifications are documented in the Appendix.

Benchmark Results

The benchmarking experiment produced consistent results across repeated runs under both cold and warm cache conditions. Surprisingly, **CSV files outperformed Parquet files** in terms of load time and memory efficiency in the given test environment.

The table below summarizes average file loading time and memory usage across formats:

CACHE TYPE	FORMAT	AVG TIME (SEC)	AVG MEMORY (MB)
COLD	CSV	0.29	32.6
COLD	Parquet	0.82	205.1
WARM	CSV	0.25	14.8
WARM	Parquet	0.82	132.9

Table 1. Average load time and memory usage under different cache conditions.

As shown above, CSV demonstrated 3-4x faster load times than Parquet, with memory usage reduced by up to **85%** in warm cache conditions. These findings run counter to the widespread belief that Parquet is always the more efficient format.

A visual representation of load time differences is shown in the following chart:

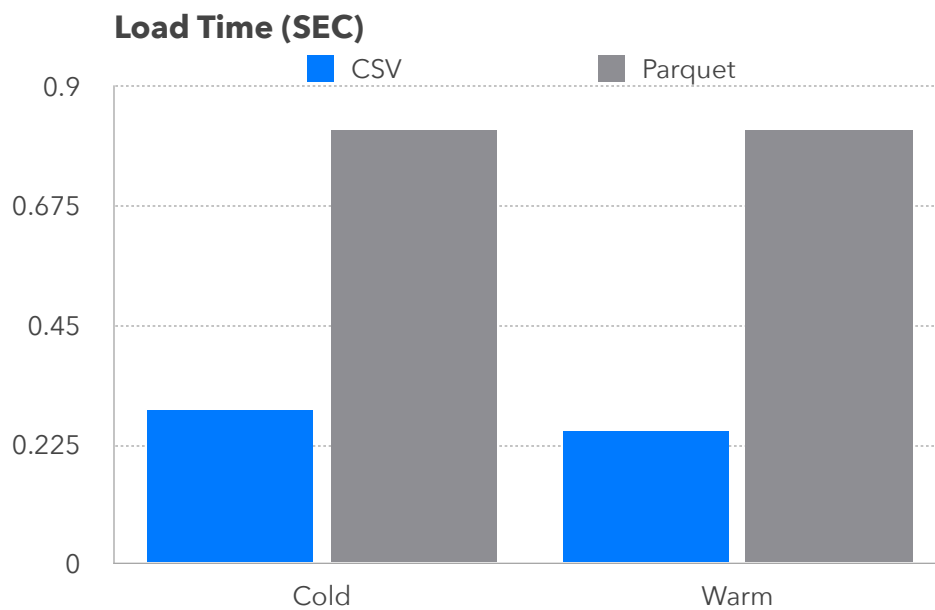


Figure 1. Average file load time (in seconds) for CSV and Parquet formats under cold and warm cache conditions. CSV files show significantly faster loading performance compared to Parquet.

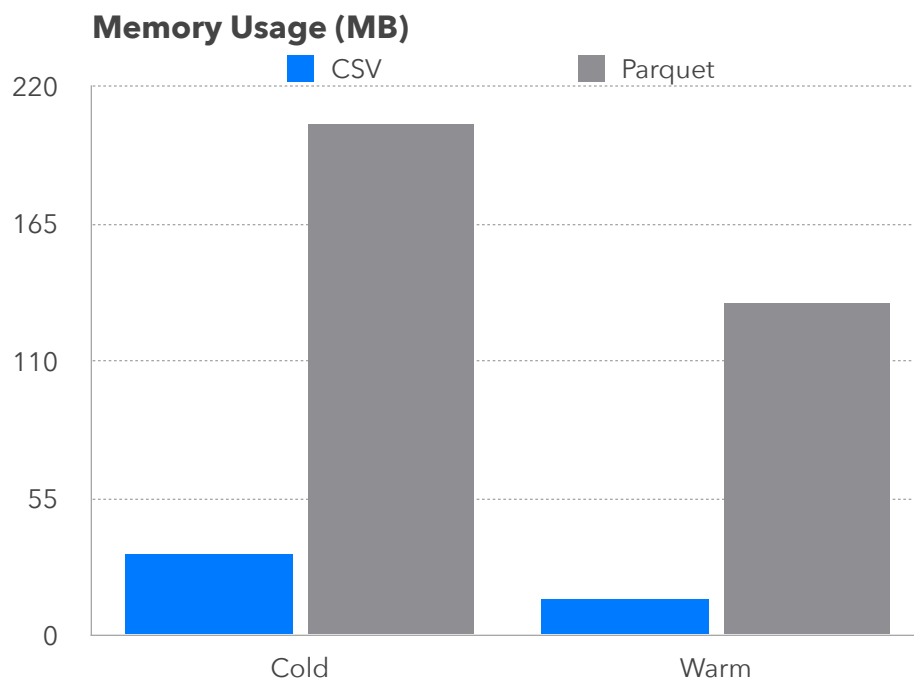


Figure 2. Average memory usage (in MB) for CSV and Parquet files under cold and warm cache conditions. CSV consistently uses less memory than Parquet, particularly under warm cache scenarios.

Profiling Analysis

To further investigate the performance gap between CSV and Parquet files, we performed a function-level runtime profiling using Python's `cProfile` module, with the results visualized through `SnakeViz` in icicle view.

The profiling focused on `read_csv().collect()` and `read_parquet().collect()` calls within the benchmarking loop. As shown in the visualization below, a substantial portion of the total execution time during Parquet file loading was consumed by the `collect()` method of a `PyLazyFrame` object.

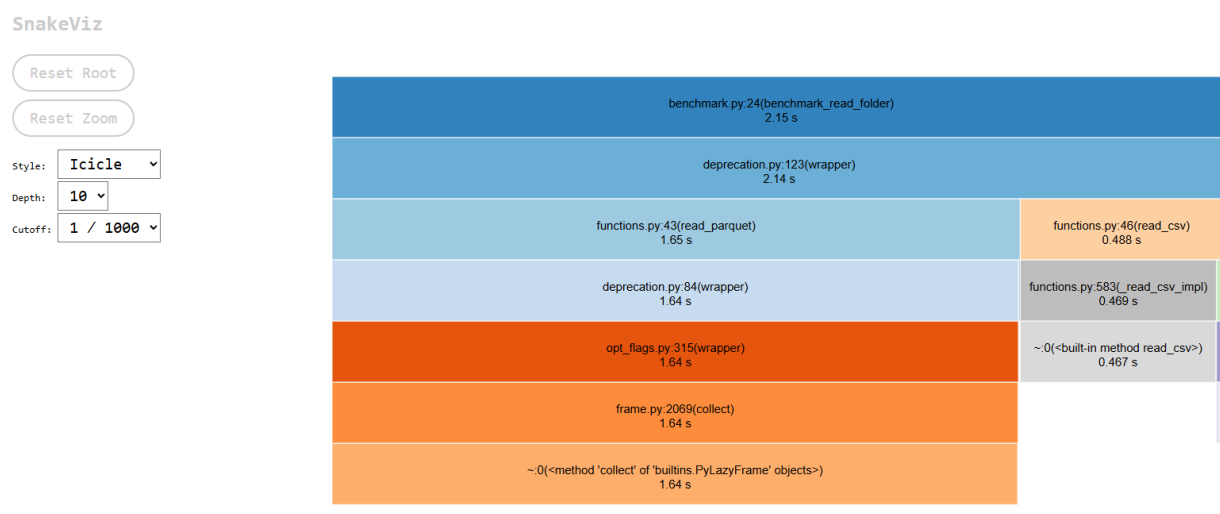


Figure 3. `SnakeViz` profiling visualization (icicle view) showing time distribution for Parquet and CSV loading operations.

The key findings are:

- **Parquet I/O heavily relies on lazy execution**, with the `collect()` call alone accounting for **~1.64 seconds**, dominating the entire process.

- In contrast, CSV loading completed significantly faster, with the total time for ``read_csv`` including all internal calls (e.g., ``_read_csv_impl``) staying under **0.49 seconds**.
- This suggests that **lazy evaluation**, while efficient in many analytical workloads, can introduce overhead in simple I/O operations where minimal downstream transformation occurs.
- Additional time appears to be spent in compatibility wrappers (e.g., ``deprecation.py``, ``opt_flags.py``), further inflating Parquet's total loading time.

These insights reinforce the benchmark findings: despite Parquet's reputation for speed and compression, in lightweight local environments and small files, CSV loading can be both faster and more memory-efficient.

Insights & Discussion

The benchmarking results revealed a notable and somewhat counterintuitive finding: CSV files consistently outperformed Parquet files in both load time and memory efficiency under the tested conditions. This stands in contrast to the commonly held belief that Parquet, a columnar and compressed format, is inherently more efficient for analytical workflows.

A deeper inspection suggests that CSV's advantage stems from its simpler structure and lower overhead during I/O operations. While Parquet offers advanced features like schema enforcement and column pruning, these benefits are accompanied by additional decoding and decompression costs at runtime. In the profiling data visualized via `SnakeViz`, it is evident that the `read_parquet()` function involved multiple wrapper calls and consumed more cumulative processing time compared to `read_csv()`, which followed a relatively direct execution path. This layered processing contributed significantly to Parquet's longer load times.

The cache conditions also influenced the performance profile. In cold cache scenarios, the lack of file system caching magnified the performance gap between formats, emphasizing Parquet's initial overhead. Under warm cache conditions, CSV further benefited from repeated reads, achieving up to **85% reduction** in memory usage compared to Parquet. These results indicate that CSV's leaner structure better leverages cached reads and avoids redundant processing, especially in short-lived, localized tasks.

Importantly, these findings do not contradict Parquet's advantages in all contexts. Parquet is still preferable for distributed, columnar analytics over large datasets. However, in **local data exploration or scripting environments**,

where speed and memory efficiency are critical under tight feedback loops, CSV may be the more pragmatic choice.

Ultimately, this study challenges the default assumption that Parquet is always superior. Instead, it highlights the need to consider workload type, system environment, and toolchain behavior when choosing a file format for data analysis.

Recommendation & Best Practices

The results of this benchmark underscore a fundamental principle in data engineering: **there is no one-size-fits-all file format**. Rather, file format selection should align with the specific context, constraints, and goals of the task at hand.

Based on the findings of this analysis, we offer the following recommendations:

When to Use CSV

- **Rapid local analysis:** Ideal for short-term projects, prototyping, and Jupyter-based exploration.
- **Lower memory footprint:** Particularly beneficial under warm cache conditions.
- **Minimal dependencies:** Suitable for environments with limited toolchain support or where simplicity is preferred.
- **Batch loading of small files:** When reading many small, similarly structured files, CSV shows lower overhead.

When to Use Parquet

- **Large-scale, columnar workloads:** Best for distributed processing (e.g., Spark, Dask) and big data pipelines.
- **Selective column access:** Parquet enables partial reads, reducing I/O in wide tables.

- **Long-term data storage:** Its schema enforcement and compression make it optimal for data lakes.
- **Cross-platform compatibility:** Preferred in cloud-native ecosystems (AWS, GCP, Azure).

Best Practices

- **Benchmark in your own environment:** Performance is highly dependent on hardware, cache state, and workload shape.
- **Profile before scaling:** Use tools like `cProfile` and `SnakeViz` to identify runtime bottlenecks early.
- **Favor explicitness over assumptions:** Avoid choosing file formats based solely on reputation or defaults.

Ultimately, understanding the trade-offs between performance, memory, and usability enables smarter, more efficient workflows. Always let **data behavior guide engineering choices**—not dogma.

Appendix A. Hardware & Environment Specs

COMPONENT	SPECIFICATION
OS	Windows 10 Pro, Version 22H2, Build 19045.5854
CPU	AMD Ryzen 5 5600X – 6C/12T @ 3.70 GHz
RAM	15.92 GB
GPU	NVIDIA GeForce RTX 3060
PYTHON	3.13.3 (64-bit)
LIBRARIES	Polars, Pandas, Psutil, Matplotlib, SnakeViz, gprof2dot

Appendix B. Benchmark Script Snippet

```
import polars as pl
import time

def benchmark_file_loading(path, fmt):
    if fmt == "csv":
        start = time.perf_counter()
        df = pl.read_csv(path)
        elapsed = time.perf_counter() - start
    elif fmt == "parquet":
        start = time.perf_counter()
        df = pl.read_parquet(path).collect()
        elapsed = time.perf_counter() - start
    return elapsed, df
```

Appendix C. On-Disk File Size Comparison Between CSV and Parquet Formats

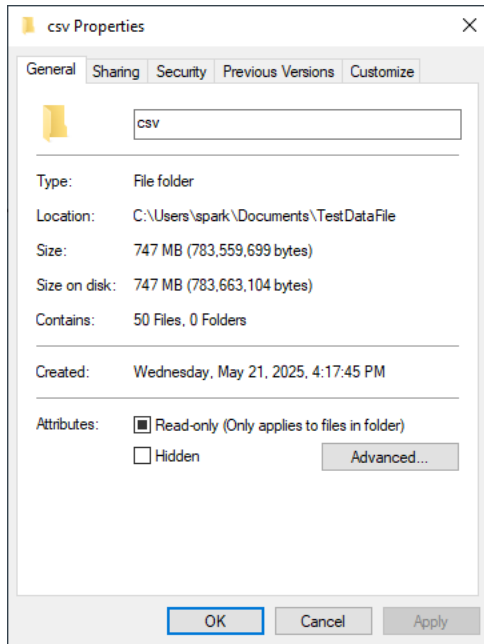


Figure C1. Total disk size of 50 CSV files used in the benchmark experiment (747 MB).

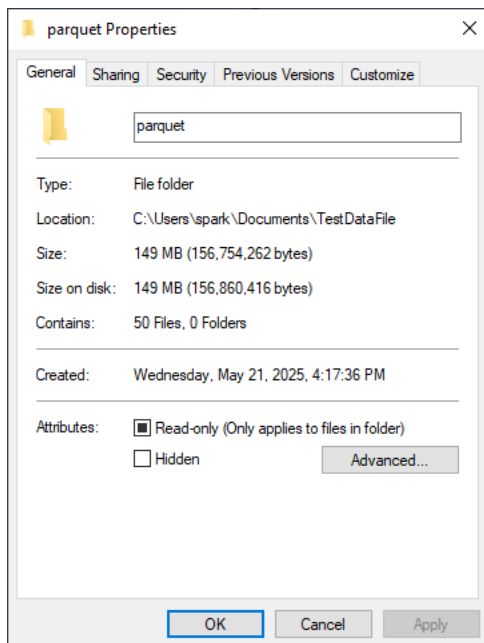


Figure C2. Total disk size of 50 Parquet files used in the benchmark experiment (149 MB).

As shown, the Parquet dataset occupies approximately 80% less disk space than the equivalent CSV dataset, highlighting its compression efficiency on disk.

• • • *End of Report* • • •